

# Diffusion Models

## Schedule

$\beta_t$  refers to something called **schedule** and values can range from 0 to 1. The values are usually kept low to prevent variance from exploding. The 2020 paper[2] uses a **linear schedule** so the output looks like that:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t, \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$



Figure 3: Entire forward diffusion process using linear schedule with 1000 time steps. (Source: erdem.pl)

Researchers from OpenAI in their 2021 paper[3] decided that using a linear schedule is not that efficient. As you've seen before, most of the information from the original image is lost after around half of the total steps. They designed their own schedule and called it the **cosine schedule** (Fig. 5). The improvement in schedule allowed them to reduce the number of steps to 50.



## FORWARD DIFFUSION

# The schedule decides how fast the image dies

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \varepsilon$$

$$\sqrt{\bar{\alpha}_t}$$

how much image is left

$$\sqrt{(1 - \bar{\alpha}_t)}$$

how much noise is in

Choosing  $\beta_t$  chooses the whole curve of  $\sqrt{\bar{\alpha}_t}$ .

Two standard choices: **linear** and **cosine**.

## SCHEDULE 1

# Linear

Space  $\beta$  evenly from start to finish.

$\beta_t$  from  $1 \times 10^{-4}$  to 0.02, evenly

$\beta$  at  $t = 1$

**0.0001**

$\beta$  at  $t = 500$

**0.0101**

$\beta$  at  $t = 1000$

**0.0200**

Simple — but it is  $\beta$  that is linear, **not** the signal.

## SCHEDULE 2

# Cosine

Choose the **signal curve** first, then derive  $\beta$ .

$$\bar{\alpha}_t = \cos^2\left( (t/T + s) / (1 + s) \cdot \pi/2 \right) \quad , \quad s = 0.008$$

$$\beta_t = 1 - \bar{\alpha}_t / \bar{\alpha}_{t-1} \quad ( \text{ capped at } 0.999 )$$

The cosine falls gently at both ends and steadily in the middle.

$\beta$  is now an **output** of the design, not the input.

## SIDE BY SIDE

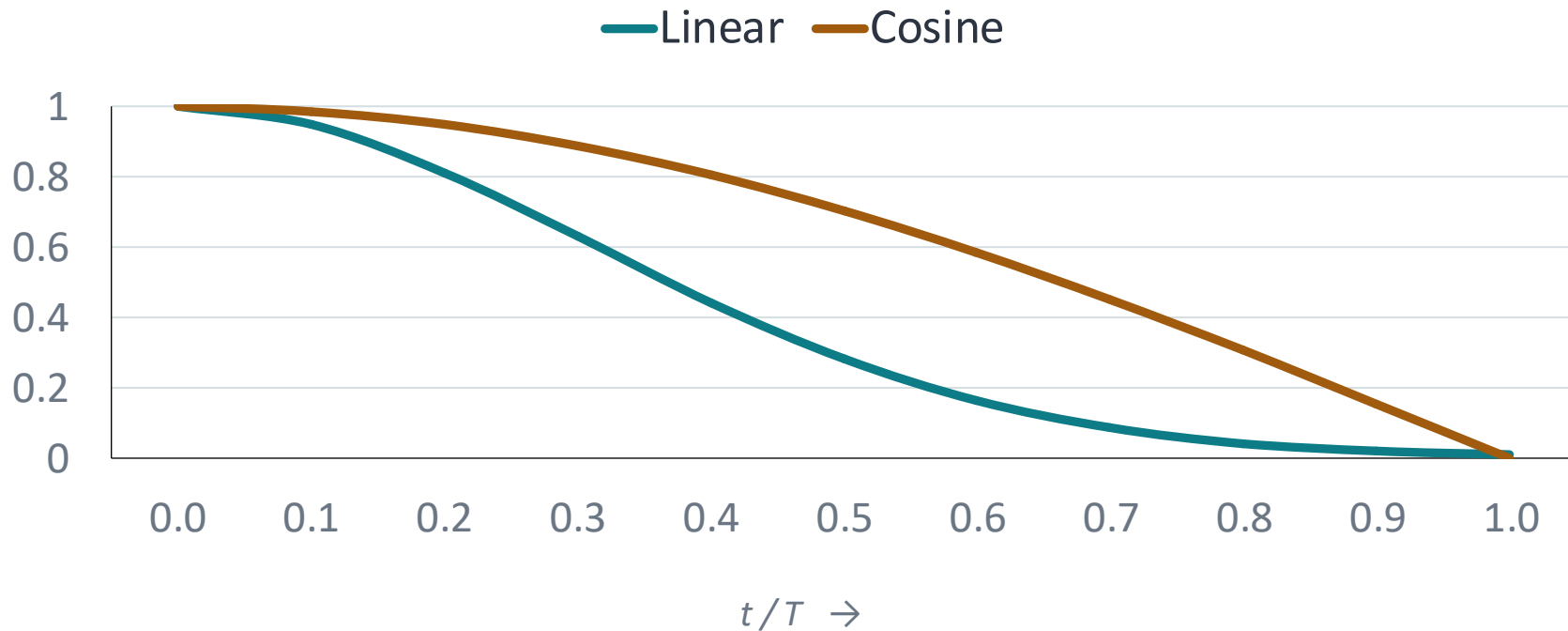
# How much image is left at each stage

| $t / T$ | LINEAR $\bar{\alpha}t$ | LINEAR $\sqrt{\alpha}t$ | COSINE $\bar{\alpha}t$ | COSINE $\sqrt{\alpha}t$ |
|---------|------------------------|-------------------------|------------------------|-------------------------|
| 0.0     | 0.9999                 | 1.000                   | 1.0000                 | 1.000                   |
| 0.2     | 0.6563                 | 0.810                   | 0.8978                 | 0.948                   |
| 0.4     | 0.1936                 | 0.440                   | 0.6460                 | 0.804                   |
| 0.5     | 0.0778                 | 0.279                   | 0.4923                 | 0.702                   |
| 0.6     | 0.0256                 | 0.160                   | 0.3393                 | 0.583                   |
| 0.8     | 0.0015                 | 0.039                   | 0.0931                 | 0.305                   |
| 1.0     | 0.0000                 | 0.006                   | 0.0000                 | 0.000                   |

Half way through, linear has 28% of the image left — cosine has 70%.

SIDE BY SIDE

## The same numbers as a curve



Linear collapses early; cosine falls at a steady rate.

## WHY IT MATTERS

# Linear wastes a third of its steps

Steps where the signal is already below 10% — nothing left to learn from.

LINEAR

33%

*signal gone by  $t = 674$*

COSINE

6%

*signal gone by  $t = 936$*

Nichol & Dhariwal could skip **20% of the reverse steps** of a linear model with almost no loss in FID.

## REFERENCES

# Where these come from

### LINEAR SCHEDULE

Ho, Jain & Abbeel (2020). “Denoising Diffusion Probabilistic Models.” NeurIPS 33.

### COSINE SCHEDULE

Nichol & Dhariwal (2021). “Improved Denoising Diffusion Probabilistic Models.” ICML 38.

## EXERCISE

# Forward diffusion on a $2 \times 2$ image

THE IMAGE  $x_0$

|     |      |
|-----|------|
| 0.8 | -0.6 |
| 0.4 | 1.0  |

THE SCHEDULE

$$\beta_1 = 0.19$$

$$\beta_2 = 0.36$$

$$\beta_3 = 0.51$$

Pixel values are scaled to  $[-1, 1]$ .

THE NOISE DRAWN AT EACH STEP

$\epsilon_1$

|     |      |
|-----|------|
| 1.0 | -1.0 |
| 0.5 | 0.5  |

$\epsilon_2$

|      |      |
|------|------|
| -0.5 | 1.0  |
| 1.0  | -1.0 |

$\epsilon_3$

|      |     |
|------|-----|
| 0.5  | 0.5 |
| -1.0 | 1.0 |

$\bar{\epsilon}$

|     |      |
|-----|------|
| 1.0 | -0.5 |
| 0.5 | -1.0 |

## EXERCISE

# What to compute

$$\alpha_t = 1 - \beta_t \quad \bar{\alpha}_t = \prod_{s \leq t} \alpha_s$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{(1 - \bar{\alpha}_t)} \varepsilon$$

(a)

Find  $\alpha$  and  $\bar{\alpha}$

(b)

Get  $x_3$  in one shot

## SOLUTION · (a)

# The schedule

| t | $\beta_t$ | $\alpha_t$ | $v\alpha_t$ | $\bar{\alpha}_t$ | $v\bar{\alpha}_t$ |
|---|-----------|------------|-------------|------------------|-------------------|
| 1 | 0.19      | 0.81       | 0.900       | 0.81             | 0.900             |
| 2 | 0.36      | 0.64       | 0.800       | 0.5184           | 0.720             |
| 3 | 0.51      | 0.49       | 0.700       | 0.2540           | 0.504             |

$v\bar{\alpha}_t$  is just the product of the  $v\alpha_s$ :  $0.9 \times 0.8 \times 0.7 = 0.504$

Signal weight falls **0.900  $\rightarrow$  0.720  $\rightarrow$  0.504**

## SOLUTION · (b)

# The one-shot closed form

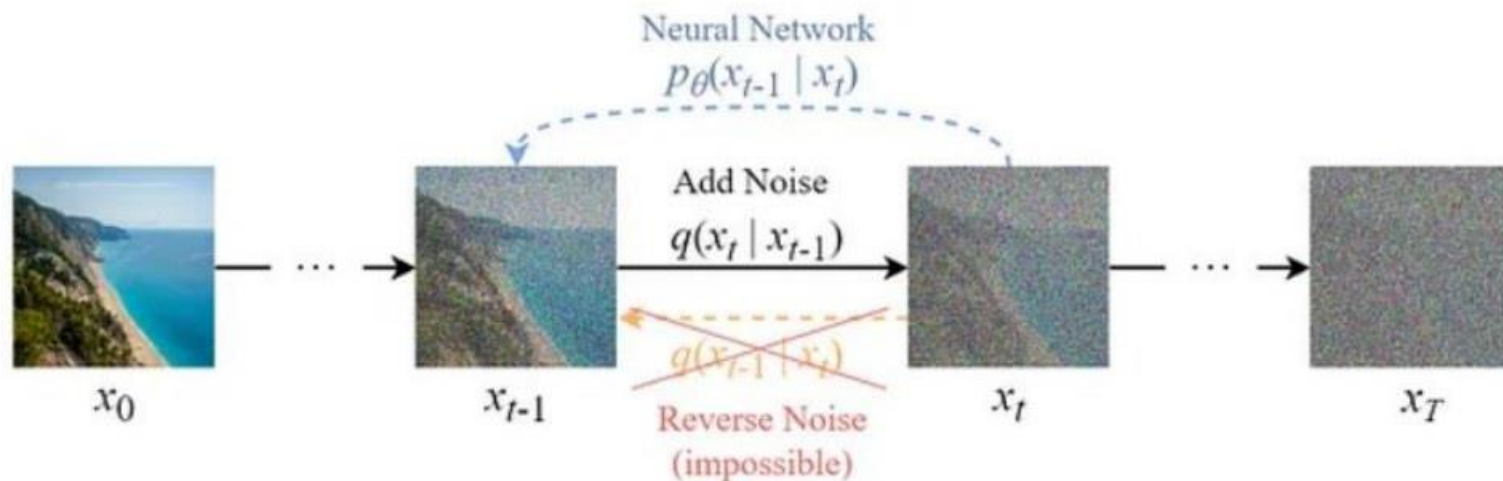
$$x_3 = 0.504 \cdot x_0 + 0.864 \cdot \bar{\epsilon}$$

| $x_0$                                                                              |       | $\bar{\epsilon}$ |     | $x_3$ |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
|------------------------------------------------------------------------------------|-------|------------------|-----|-------|---|-------------------------------------------------------------------------------------|-----|------|-----|------|---|-----------------------------------------------------------------------------------------|------|-------|------|-------|
| <table><tr><td>0.8</td><td>-0.6</td></tr><tr><td>0.4</td><td>1.0</td></tr></table> | 0.8   | -0.6             | 0.4 | 1.0   | + | <table><tr><td>1.0</td><td>-0.5</td></tr><tr><td>0.5</td><td>-1.0</td></tr></table> | 1.0 | -0.5 | 0.5 | -1.0 | = | <table><tr><td>1.27</td><td>-0.73</td></tr><tr><td>0.63</td><td>-0.36</td></tr></table> | 1.27 | -0.73 | 0.63 | -0.36 |
| 0.8                                                                                | -0.6  |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
| 0.4                                                                                | 1.0   |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
| 1.0                                                                                | -0.5  |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
| 0.5                                                                                | -1.0  |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
| 1.27                                                                               | -0.73 |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |
| 0.63                                                                               | -0.36 |                  |     |       |   |                                                                                     |     |      |     |      |   |                                                                                         |      |       |      |       |

top-left:  $0.504(0.8) + 0.864(1.0) = 1.267$

No iteration needed — any  $x_t$  comes straight from  $x_0$ .

# Reverse Diffusion Process



Target Distribution

$$q(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

Approximated Distribution

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Learnable parameters  
(Neural Network)

Unlike the forward process, we cannot use  $q(x_{t-1}|x_t)$  to reverse the noise since it is intractable (uncomputable).

Thus we need to train a neural network  $p_\theta(x_{t-1}|x_t)$  to approximate  $q(x_{t-1}|x_t)$ . The approximation  $p_\theta(x_{t-1}|x_t)$  follows a normal distribution and its mean and variance are set as follows:

$$\begin{cases} \mu_\theta(x_t, t) &:= \tilde{\mu}_t(x_t, x_0) \\ \Sigma_\theta(x_t, t) &:= \tilde{\beta}_t I \end{cases}$$

mean and variance of  $p_\theta$

## Loss Function

We can define our loss as a Negative Log-Likelihood:

$$\text{Loss} = -\log(p_{\theta}(x_0))$$

Depends on  $x_1, x_2, \dots, x_T$   
Therefore it is intractable!

Negative log-likelihood

This setup is very similar to the one in VAE. instead of optimizing the intractable loss function itself, we can optimize the Variational Lower Bound.

Detailed ELBO derivation: [https://lilianweng.github.io/posts/2021-07-11-diffusion-models/?utm\\_source=chatgpt.com](https://lilianweng.github.io/posts/2021-07-11-diffusion-models/?utm_source=chatgpt.com)

$$\min_{\theta} \mathbb{E}_{x_0 \sim p(x_0)} \left[ -\log p_{\theta}(x_0|x_1) + \underbrace{KL(p(x_T|x_0)||p_{\theta}(x_T))}_{\text{blue oval}} + \sum_{t=2}^T \underbrace{KL(p(x_{t-1}|x_t, x_0)||p_{\theta}(x_{t-1}|x_t))}_{\text{green oval}} \right]$$

This term can be ignored because  $p_{\theta}(x_T)$  is  $\mathcal{N}(0, I)$  and does not depend on  $\theta$ .

At each time step  $t$ ,  $p_{\theta}(x_{t-1}|x_t)$  is as close as possible to the true posterior of the forward process when conditioned on the original image.

# DDPMs. Training Objective. Simplifications

$$\min_{\theta} \mathbb{E}_{x_0 \sim p(x_0)} \left[ -\log p_{\theta}(x_0|x_1) + \sum_{t=2}^T KL(p(x_{t-1}|x_t, x_0) || p_{\theta}(x_{t-1}|x_t)) \right]$$

This term is handled separately (using reconstruction error) or omitted in the simplified objective.

The KL divergence between two Gaussian distributions is proportional to the squared L<sub>2</sub> distance between their means when they have the same covariance.

$$D_{KL}(p||q) = \frac{1}{2\sigma^2} \|\mu_p - \mu_q\|_2^2.$$

# Where the mean formula comes from

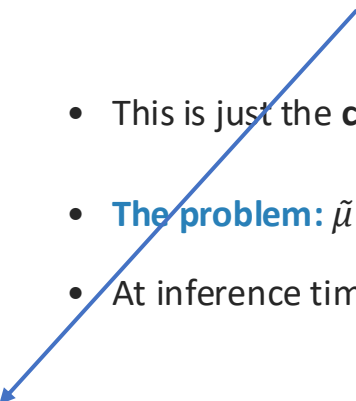
- We want  $x_{t-1}$  given  $x_t$ . The true posterior — if we knew the clean image — is Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I)$$

- where:

$$\tilde{\mu}(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

- This is just the **conditional mean of two Gaussians** (derivable using Bayes rule).
- **The problem:**  $\tilde{\mu}$  depends on  $x_0$  — the clean image we are trying to generate.
- At inference time we do not have  $x_0$ . That is the whole point of sampling.



Derived using pdf of the Gaussian distribution and Bayes' rule. Details given at following link:  
[https://lilianweng.github.io/posts/2021-07-11-diffusion-models/?utm\\_source=chatgpt.com](https://lilianweng.github.io/posts/2021-07-11-diffusion-models/?utm_source=chatgpt.com)

# The trick: recover $x_0$ from the predicted noise

- The forward process has a closed form — it is exact, for any  $t$ :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

- It contains  $x_0$ , so simply rearrange it:

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

- We do not know the true  $\epsilon$  either — but the UNet predicts it. Substitute  $\epsilon_\theta(x_t, t)$ :

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

- $\hat{x}_0$  is the model's current **guess** of the clean image, implied by the noise it predicted.

- We want  $x_{t-1}$  given  $x_t$ .
- The true posterior is Gaussian:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I)$$

- where:

$$\tilde{\mu}(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

- This is just the **conditional mean of two Gaussians** (derivable using Bayes rule).

Put  $\hat{x}_0$  into  $\tilde{\mu}(x_t, x_0)$ .

After algebraic simplification (see Ho et al., 2020 Appendix C), you get:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right)$$

Derivation is also given in Appendix A of this lecture

- This is the **mean formula** used in reverse process.

# DDPMs: Training Objective Simplifications

$$\begin{aligned}
 L_t &= \mathbb{E}_{x_0, \epsilon} \left[ \frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t(x_t) - \mu_\theta(x_t, t) \right\|^2 \right] \\
 &= \mathbb{E}_{x_0, \epsilon} \left[ \frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) - \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \right\|^2 \right] \\
 &= \mathbb{E}_{x_0, \epsilon} \left[ \frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left[ \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) - \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \right] \right\|^2 \right]
 \end{aligned}$$

take the common factor  $1/\sqrt{\alpha_t}$  outside both brackets

$$= \mathbb{E}_{x_0, \epsilon} \left[ \frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} (\epsilon_\theta(x_t, t) - \epsilon_t) \right\|^2 \right]$$

$x_t - x_t = 0$ , so only the noise terms remain; factor out  $(1 - \alpha_t)/\sqrt{1 - \bar{\alpha}_t}$

$$\|cv\|^2 = c^2\|v\|^2 \quad \text{and} \quad \|a - b\|^2 = \|b - a\|^2$$

$$= \mathbb{E}_{x_0, \epsilon} \left[ \frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t)\sigma_t^2} \|\epsilon_t - \epsilon_\theta(x_t, t)\|^2 \right]$$

ignorable



$$L_t^{\text{simple}} = \mathbb{E}_{t \sim [1, T], x_0, \epsilon_t} \left[ \|\epsilon_t - \epsilon_\theta(x_t, t)\|^2 \right]$$

It depends **only on the predefined noise schedule and the timestep  $t$** ; it does not depend on the model parameters  $\theta$ , so we can ignore it to obtain a simplified loss.

## Simplified Loss

So the final simplified training objective is as follows:

$$x_t = \sqrt{\bar{a}_t} x_0 + \sqrt{1 - \bar{a}_t} \epsilon$$

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[ \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2 \right]$$

Simplified training objective

**Loss Function:** The UNet is trained to minimize the difference between its predicted noise  $\hat{\epsilon}_{\theta}$  and the actual noise  $\epsilon$  (which was added during forward diffusion):

$$L = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \hat{\epsilon}_{\theta}(x_t, t)\|^2]$$

# DDPMs. Training Algorithm

---

## Algorithm 1 Training

---

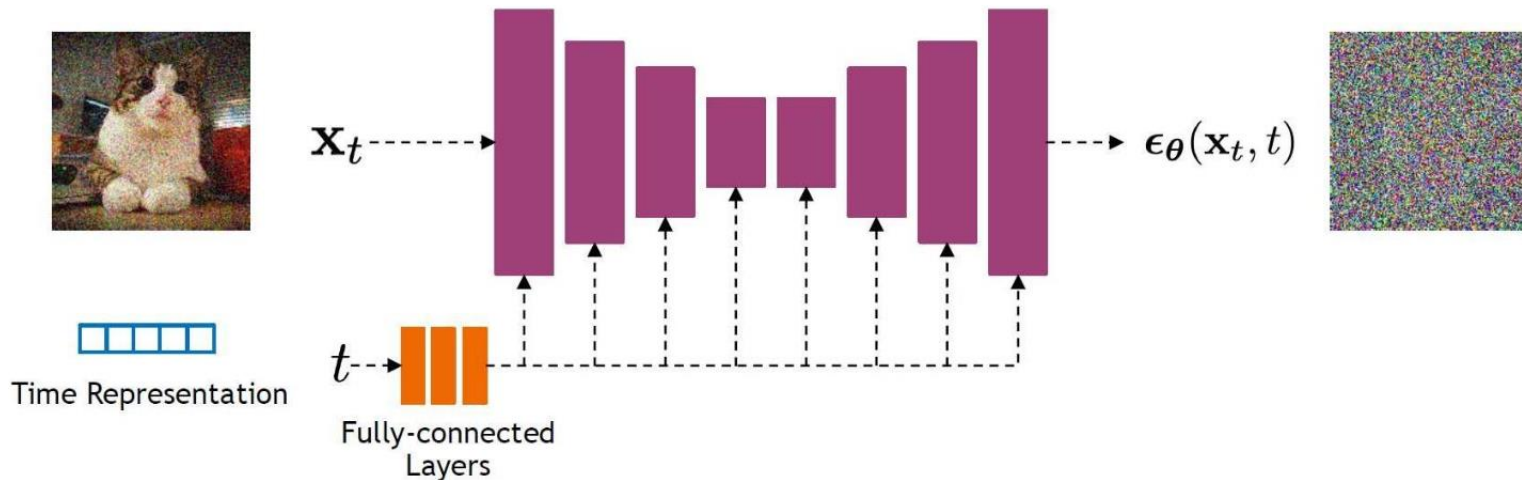
- 1: **repeat**
  - 2:  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
  - 3:  $t \sim \text{Uniform}(\{1, \dots, T\})$
  - 4:  $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  - 5: Take gradient descent step on  
$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
  - 6: **until** converged
- 

This is  $\mathbf{x}_t$  calculated through the forward pass equation by adding noise



# Implementation Considerations

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent  $\epsilon_{\theta}(\mathbf{x}_t, t)$



# Reverse inference/sampling part

## 1. Input Noisy Image $x_t$

- Given an image  $x_t$  at time  $t$ , we want to move backward in the diffusion process to reconstruct  $x_{t-1}$ .

## 2. UNet Predicts Noise $\epsilon_\theta(x_t, t)$

- The model (usually a **UNet**) takes in  $x_t$  and estimates the noise component  $\epsilon_\theta(x_t, t)$  that was added in the forward process.

## 3. Compute the Mean $\mu_\theta$ for Transition to $x_{t-1}$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

- This tells us where  $x_{t-1}$  should be centred — it needs only  $x_t$  and the predicted noise.
- Everything on the right-hand side is already available:  $x_t$  is the current sample,  $\epsilon_\theta(x_t, t)$  is the UNet output.

The mean formula  
derived in Appendix A  
of this lecture

#### 4. Sample $x_{t-1}$ Using a Stochastic Term

- Instead of directly setting  $x_{t-1} = \mu_\theta$ , we **add Gaussian noise** to make the process probabilistic:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

- $\sigma_t$  is a predefined noise scale (depends on the variance schedule).
- $z$  is standard Gaussian noise, ensuring randomness in the process.
- If  $t = 1$ , drop the noise term — take the mean directly (final clean sample).

#### 5. Repeat Until $x_0$ is Reached

- This step-by-step denoising continues until we reach  $x_0$ , the final clean image.

**Note:** the clean image  $x_0$  never appears anywhere in this loop — only  $x_t$  and the predicted noise.

---

## Algorithm 2 Sampling

---

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)}_{\text{mean}} + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

---

The mean formula  
derived in Appendix A  
of this lecture

$\sigma_t$  is the standard deviation of the  
noise we add back at this step. It is  
**not learned**. It is built from the  
same  $\beta$  schedule as the forward  
process

The added noise  
helps maintain  
diversity in generated  
images.

**At each step:** take your current noisy image  $x_t$ , feed it to the UNet with its timestamp  $t$ , predict noise, compute mean, sample  $x_{t-1}$  from that mean — then repeat with this new  $x_{t-1}$  for  $t - 1$ . Keep going until  $t = 1$ , at which point you get  $x_0$  (final clean image).

- **You don't jump straight from noise to a clean image.**

You take **T tiny steps**, each one making the image a little cleaner.

- The **UNet is reused at every step**, it knows  $t$  because you give it as an input (positional/time embedding).
- Each step is **probabilistic** because of the random Gaussian you add (except final step where no randomness is added).

Diffusion models are slower than GANs — **you need to do this T times** (e.g.,  $T=1000$  for DDPM, though improved samplers like DDIM can do it in far fewer steps)

# Why do we add noise when denoising?

- The forward diffusion process was **stochastic** (random noise added at each step).
  - To reverse it, we also need a **stochastic process** rather than a deterministic one.
  - The added noise helps maintain diversity in generated images.
- 
- Creativity and Variation:** It allows the model to generate creative and varied outputs, even when starting from the same latent noise. Imagine generating different variations of a "cat" image – different poses, colors, fur patterns, etc.
  - Avoiding Overfitting:** Stochasticity helps prevent the model from memorizing specific training examples and encourages it to learn the underlying distribution of the data.
  - Exploring the Latent Space:** The added noise allows the reverse process to explore different regions of the latent space, leading to a wider range of generated samples.

# The U-Net Model

## Dataset

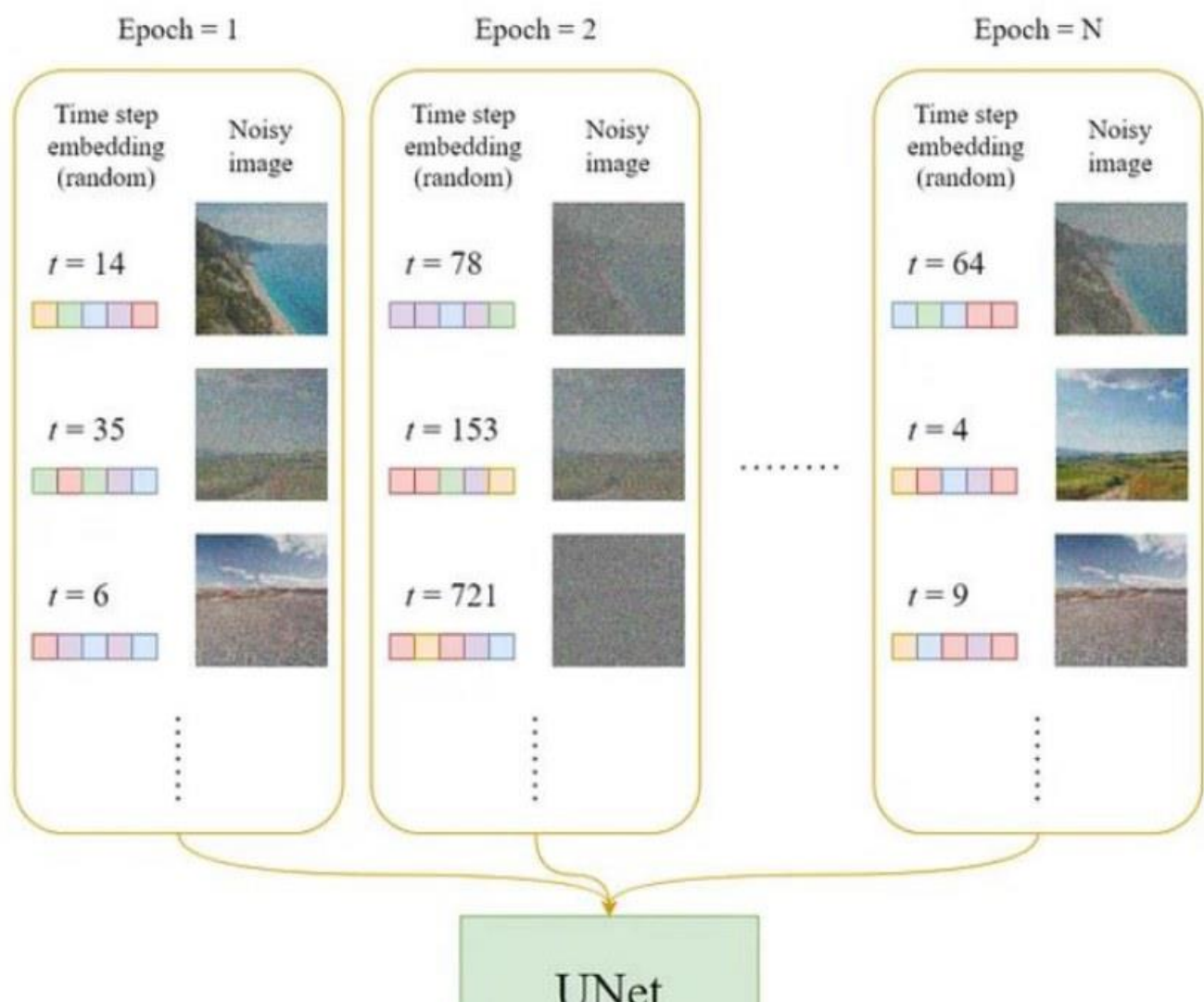
In each epoch:

1. A random time step  $t$  will be selected for each training sample (image).
2. Apply the Gaussian noise (corresponding to  $t$ ) to each image.
3. Convert the time steps to embeddings (vectors).

•**Training Goal:** The goal of training is to teach the U-Net to predict the noise  $\epsilon$  at any given timestep  $t$ .

• The network needs to be able to handle all noise levels, regardless of the order in which they are presented.

Why do we pass random time steps in Unet during training?



Why do we pass random time steps in Unet during training?

- Efficiency and Generalization:** Training with random timesteps is more efficient and leads to better generalization. If the U-Net were only trained on sequential timesteps, it might overfit to that specific sequence and not perform well on other timesteps or noise levels it hasn't seen during training.
- Independence of Timesteps:** The noise prediction task at each timestep is relatively independent of other timesteps. The U-Net learns a mapping from (noisy image, timestep) to noise. The order in which it learns these mappings doesn't fundamentally affect its ability to perform the task.
- Simplified Training Loop:** Random timesteps simplify the training loop. You can sample a batch of images, randomly sample timesteps for each image, add noise corresponding to those timesteps, and then train the U-Net to predict the added noise. This is much simpler than managing a sequential process during training.

During training we **simulate the forward process** to create noisy samples, and then train the UNet to predict the noise we added.

We **do NOT run the reverse process** during training

### Algorithm 1 Training

- 1: **repeat**
- 2:  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3:  $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4:  $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on  

$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$$
- 6: **until** converged

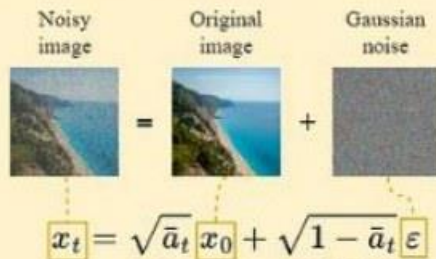
During inference, the **forward process is skipped**—we directly sample noise and denoise step by step.

For each training step:

1. Randomly select a time step & encode it



2. Add noise to image



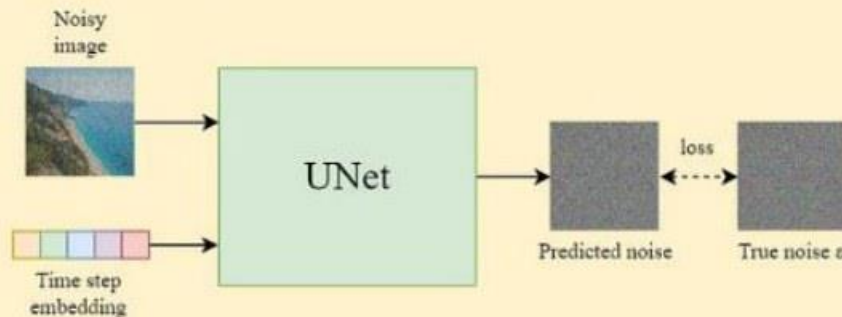
$$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

Adjust the amount of noise according to the time step  $t$

3. Train the UNet



## Algorithm 2 Sampling

- 1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for**  $t = T, \dots, 1$  **do**
- 3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$
- 4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return**  $\mathbf{x}_0$

### 1. Sample a Gaussian noise

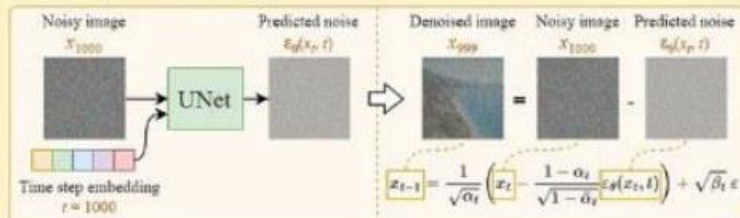
$$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\text{E.g. } T = 1000$$

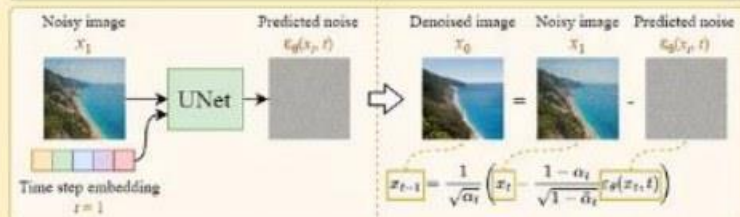
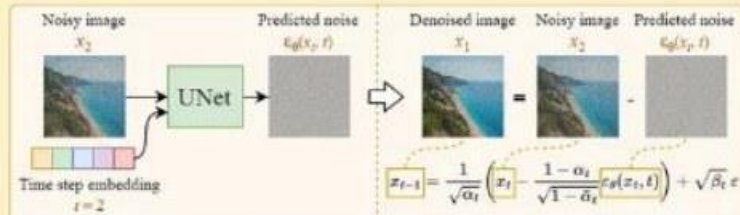
$$\mathbf{x}_{1000} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$



### 2. Iteratively denoise the image



⋮



### 3. Output the denoised image

Denoised image  $\mathbf{x}_0$



## Summary

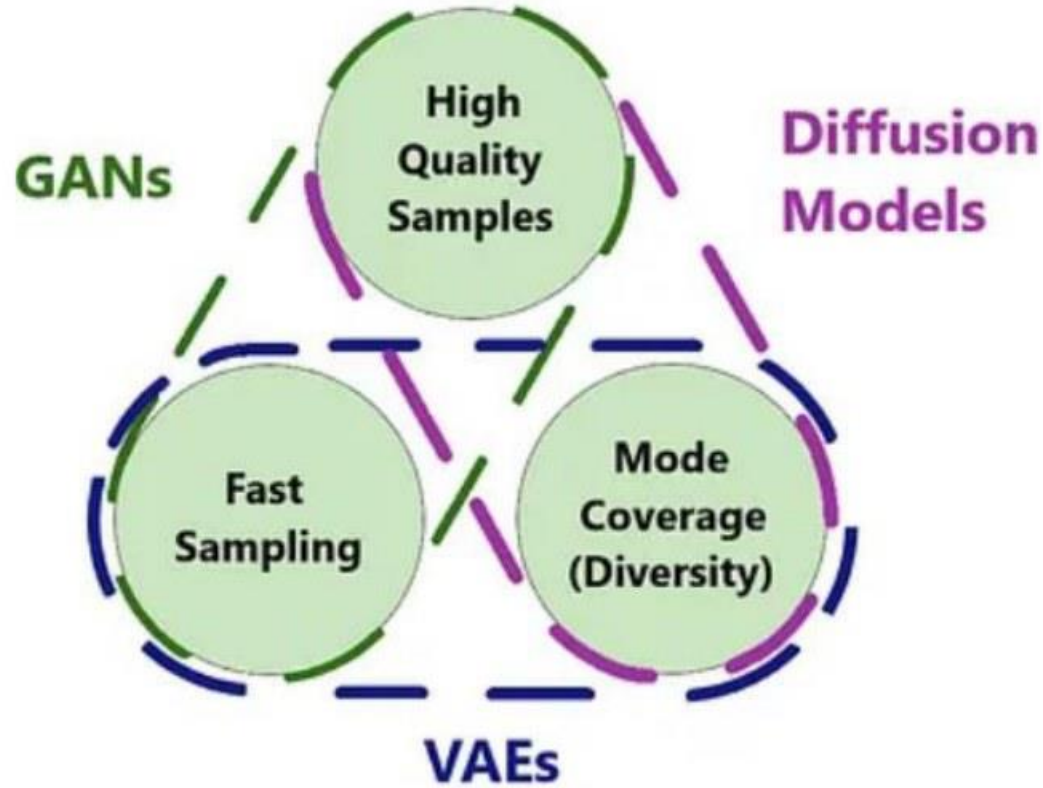
Here are some main takeaways from this article:

- The Diffusion model is divided into two parts: forward diffusion and reverse diffusion.
- The forward diffusion can be done using the closed-form formula.
- The backward diffusion can be done using a trained neural network.
- To approximate the desired denoising step  $q$ , we just need to approximate the noise  $\epsilon_t$  using a neural network  $\epsilon\theta$ .
- Training on the simplified loss function yields better sample quality.

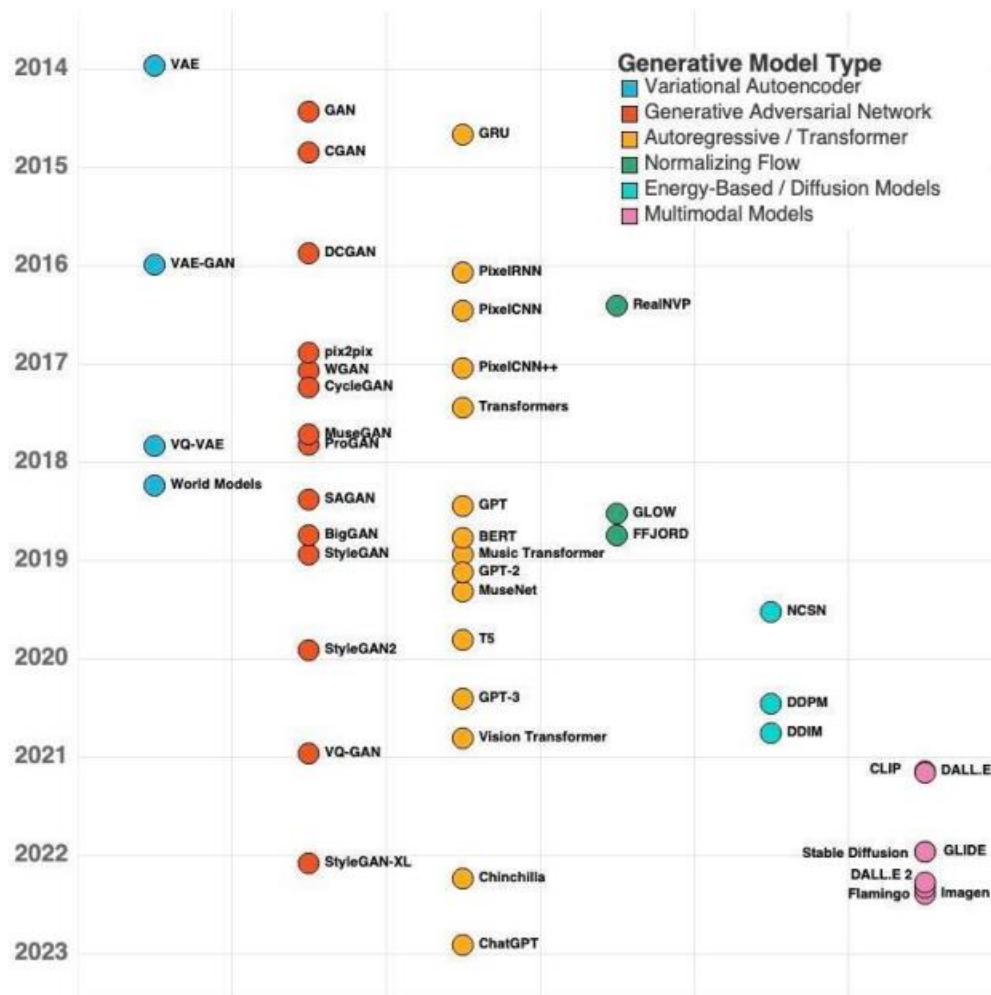
- The Diffusion process consists of **forward diffusion** and **reverse diffusion**
- Forward diffusion is used to add noise to the input image using a **schedule**
- There are different types of schedules (we've used linear), and they decide how much noise is added at the given step  $t$
- We don't have to use an iterative process to add noise, it can be done in one step
- Reverse diffusion consists of multiple steps in which a **small amount of noise is removed at every step**
- **Diffusion model predicts the entire noise**, not the difference between step  $t$  and  $t-1$
- You might use a different schedule for inference than for training (including a different number of steps)
- Diffusion model uses a **modified U-Net architecture**
- To add information about the timestep, we use sinusoidal encoding

# Training vs Inference

| Phase     | What We Do                                                                   | Goal                                                   |
|-----------|------------------------------------------------------------------------------|--------------------------------------------------------|
| Training  | Pick random $t$ , add noise (forward process), train UNet to predict noise   | Learn a <i>denoiser</i> that works for any noise level |
| Inference | Start from noise and iteratively remove noise step by step (reverse process) | Generate a clean image                                 |



# Generative AI Timeline



# References

Jonathan Ho, Ajay Jain, Pieter Abbeel. "Denoising Diffusion Probabilistic Models." NeurIPS, 2020.

<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/#reverse-diffusion-process>

<https://theaisummer.com/diffusion-models/>

<https://medium.com/@steinsfu/diffusion-model-clearly-explained-cd331bd41166>

# Appendix A

- We want  $x_{t-1}$  given  $x_t$ .
- The true posterior is Gaussian:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I)$$

- where:

$$\tilde{\mu}(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

# Substitute — collecting the terms

- Write the posterior mean compactly, with  $A$  and  $B$  for the two coefficients:

$$\tilde{\mu}(x_t, x_0) = A x_0 + B x_t \quad \hat{\epsilon} \equiv \epsilon_\theta(x_t, t)$$

$$A = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t}, \quad B = \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}$$

- Only  $x_0$  is unknown — replace it with the estimate from the previous slide:

$$\mu_\theta = A \cdot \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} + B x_t$$

- Both terms now contain  $x_t$ , so collect them:

$$\mu_\theta = \left( \frac{A}{\sqrt{\bar{\alpha}_t}} + B \right) x_t - \frac{A \sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \hat{\epsilon}$$

- Two coefficients are left to simplify — one on  $x_t$ , one on the predicted noise.

# Coefficient of the current sample

- Split  $\sqrt{\bar{\alpha}_t} = \sqrt{\bar{\alpha}_{t-1}} \cdot \sqrt{\alpha_t}$  in  $A$ , and rescale  $B$  by  $\sqrt{\alpha_t}$  over itself:

$$\frac{A}{\sqrt{\bar{\alpha}_t}} = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{(1 - \bar{\alpha}_t) \sqrt{\bar{\alpha}_{t-1}} \sqrt{\alpha_t}} = \frac{\beta_t}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}}$$
$$B = \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} = \frac{\alpha_t (1 - \bar{\alpha}_{t-1})}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}}$$

- Same denominator now, so add the numerators:

$$\frac{A}{\sqrt{\bar{\alpha}_t}} + B = \frac{\beta_t + \alpha_t (1 - \bar{\alpha}_{t-1})}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}}$$

- The numerator collapses, using  $\beta_t = 1 - \alpha_t$  and  $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$ :

$$\beta_t + \alpha_t - \alpha_t \bar{\alpha}_{t-1} = (1 - \alpha_t) + \alpha_t - \bar{\alpha}_t = 1 - \bar{\alpha}_t$$

$$\Rightarrow \frac{A}{\sqrt{\bar{\alpha}_t}} + B = \frac{1 - \bar{\alpha}_t}{(1 - \bar{\alpha}_t) \sqrt{\alpha_t}} = \frac{1}{\sqrt{\alpha_t}}$$

- The  $(1 - \bar{\alpha}_t)$  cancels top and bottom — the whole coefficient is  $1/\sqrt{\alpha_t}$ .

# Coefficient of the noise — and the result

- Reuse  $A / \sqrt{\bar{\alpha}_t}$  from the previous slide; one factor of  $\sqrt{1 - \bar{\alpha}_t}$  cancels:

$$\frac{A\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} = \frac{\beta_t}{(1 - \bar{\alpha}_t)\sqrt{\alpha_t}} \cdot \sqrt{1 - \bar{\alpha}_t} = \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}}$$

- Put both simplified coefficients back:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{\beta_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon}$$

- Factor out  $1/\sqrt{\alpha_t}$  — and this is exactly the step-3 formula:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

- $\hat{x}_0$  has disappeared. It is a derivation device, not a step in the sampling loop.**
- Computing it explicitly gives the identical number — just extra work.
- It is materialised in code only when  $\hat{x}_0$  is clipped to  $[-1, 1]$ , or in samplers such as DDIM.